

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Інтелектуальний аналіз даних»

Практичне заняття № 01 — класифікація засобами scikit-learn

Варіант 3

Виконав: Вовкотруб Олександр Віталійович

Група: ФБ-61мн

Перевірив: доц. Сахненко Н. К.

Зміст

1	Мета роботи	2
2	Постановка задачі	2
2.1	Вибір датасета	2
3	Хід виконання	2
3.1	Крок 1. Завантаження та первинний аналіз даних	2
3.2	Крок 2. Попередня обробка даних	3
3.2.1	Службові колонки: перевірка на витік мітки	3
3.2.2	Константні колонки	4
3.2.3	Пропущені значення	4
3.2.4	Кодування категоріальних ознак	5
3.2.5	Викиди	5
3.3	Крок 3. Візуалізація та дослідження даних (EDA)	6
3.3.1	Кореляції	7
3.3.2	Гістограми	8
3.3.3	Розподіл за класами	9
3.4	Крок 4. Підготовка даних до навчання	9
3.4.1	Масштабування	10
3.5	Крок 5. Навчання моделей	10
3.6	Крок 6. Підбір гіперпараметрів	11
3.6.1	kNN — <code>n_neighbors</code>	11
3.6.2	SVM — <code>GridSearchCV</code> по <code>C</code> та <code>gamma</code>	11
3.6.3	Внесок ERC20-ознак: скільки якості дає артефакт	13
3.6.4	Decision Tree — <code>max_depth</code>	14
4	Результати	14
5	Висновки	16
6	Джерела	18

1 Мета роботи

Ознайомитися з базовим workflow задачі класифікації: від вибору та аналізу даних до навчання, налаштування, порівняння та оцінювання моделей машинного навчання засобами бібліотеки `scikit-learn`.

2 Постановка задачі

Обрати датасет для задачі класифікації та виконати шість кроків: первинний аналіз даних, попередню обробку, розвідувальний аналіз (EDA), підготовку до навчання, навчання п'яти класифікаторів (kNN, Decision Tree, SVM, Random Forest, AdaBoost) і підбір гіперпараметрів.

2.1 Вибір датасета

Обрано **Ethereum Fraud Detection** — 9 841 акаунт мережі Ethereum, розмічений за ознакою причетності до шахрайських схем (скам, Ponzi, фішинг). Вибір зумовлений відповідністю напрямку підготовки та інтересом до блокчейну: це задача криміналістики розподіленого реєстру, де ознаки обчислюються безпосередньо з графа транзакцій публічного блокчейну.

Цільова змінна `FLAG`: 0 — легітимний акаунт, 1 — шахрайський. Ознаки описують поведінку акаунта: інтенсивність транзакцій, суми надісланого й отриманого ETH, кількість унікальних контрагентів, а також окремим блоком — активність із токенами стандарту ERC20.

3 Хід виконання

Розв'язок — ноутбук `solution.ipynb`. Відтворюваність забезпечується фіксованим `random_state = 42` в усіх місцях, де є випадковість.

Наведені числа отримано на `scikit-learn 1.9.0`. Прогін того самого ноутбука в Google Colab (`scikit-learn 1.6.1`) відтворює всі висновки й більшість метрик точно; розходяться лише AdaBoost ($F1 = 0.9893$ замість 0.9905) і крос-валідаційна оцінка Random Forest в абляції — це наслідок зміни версії бібліотеки, а не випадковості.

3.1 Крок 1. Завантаження та первинний аналіз даних

Датасет має розмір $9\,841 \times 51$: 50 колонок-ознак і цільова змінна `FLAG`. З них 48 числових і 3 рядкові (`Address` і дві колонки з назвами токенів).

У частини заголовків є провідні пробіли (`Total ERC20 txns`) — їх прибрано одразу після завантаження, інакше до колонок довелося б звертатися з пробілом.

Таблиця 1: Розподіл акаунтів за класами

Клас	Кількість	Частка
0 — легітимний	7 662	77.86 %
1 — шахрайський	2 179	22.14 %

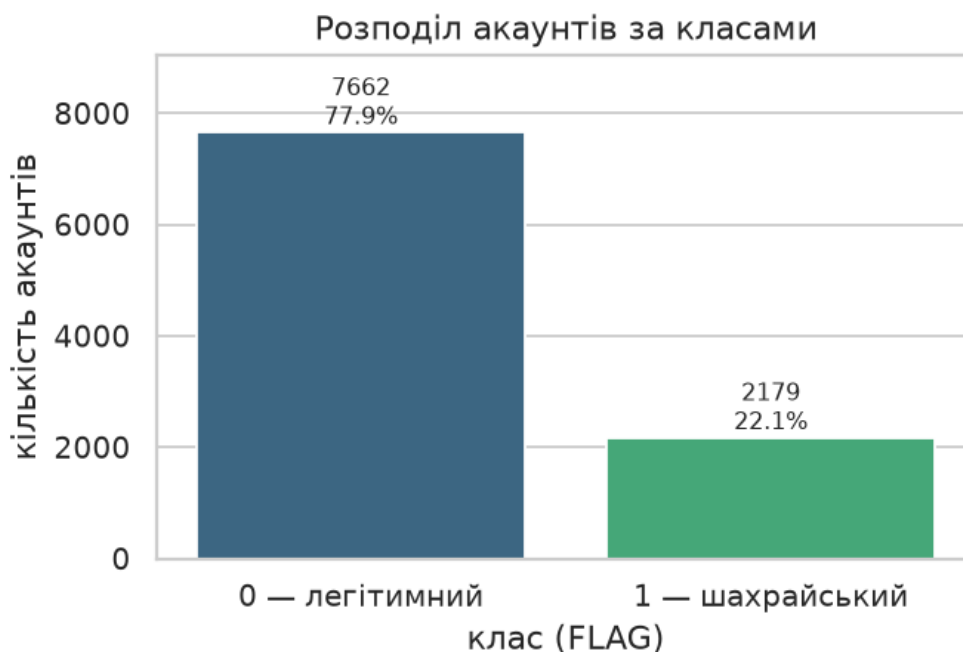


Рисунок 1: Розподіл акаунтів за класами

Класи незбалансовані у співвідношенні приблизно 1:3.5. Це помірний дисбаланс, тому oversampling чи ваги класів не обов’язкові, але **accuracy як метрика вже ненадійна**: модель, яка завжди відповідає «легітимний», отримає 0.7786. Тому основною метрикою далі є **F1-macro** — він усереднює F1 по обох класах і не дає сховати провал на меншому класі.

3.2 Крок 2. Попередня обробка даних

3.2.1 Службові колонки: перевірка на витік мітки

Три перші колонки датасета — технічні ідентифікатори: Unnamed: 0, Index і Address. Перед тим як просто їх викинути, варто перевірити, чи не «протікає» через них мітка. Перевірка виявила **грубий витік**:

Таблиця 2: Діапазони службових колонок за класами

Колонка	Клас 0	Клас 1
Unnamed: 0	0 ... 7 661	7 662 ... 9 840

Колонка	Клас 0	Клас 1
Index	1 ... 4 729	1 ... 2 179

- Unnamed: 0 — це номер рядка, а рядки **відсортовані за класом**: спочатку всі легітимні, потім усі шахрайські. Одне порівняння Unnamed: 0 >= 7662 дає стопроцентну класифікацію.
- Index нумерується **всередині кожного класу окремо**. Тому правило «Index > 2179 → легітимний» безпомилково розмічає **3 304** акаунти (частка шахраїв серед них — рівно 0.0000).
- Address — хеш гаманця, майже унікальний (9 816 значень на 9 841 рядок). Як ознака він марний, а дерево на ньому просто запам'ятало б вибірку.

Щоб оцінити масштаб, навчено дерево рішень **лише на двох службових колонках**, без жодної поведінкової ознаки:

$$F1\text{-macro} = 0.9997$$

Тобто модель, яка нічого не знає про поведінку акаунта, була б практично ідеальною. Це і є головна пастка датасета: якби ці колонки лишили «про всяк випадок», усі подальші результати були б беззмістовні, а високі метрики створювали б хибне враження якості. Усі три колонки видалено.

3.2.2 Константні колонки

Сім ERC20-колонок містять одне й те саме значення (0) для всіх без винятку рядків: ERC20 avg time between sent/rec/rec 2/contract txn та ERC20 min/max/avg val sent contract. Нульова дисперсія означає нульову інформативність, а StandardScaler на такій колонці дає би ділення на нуль. Після їх видалення лишилося 41 колонка.

3.2.3 Пропущені значення

Пропуски є у 18 колонках із 41 і утворюють рівно три групи:

Група	Колонок	Пропусків
Числові ERC20-ознаки	16	829 (одні й ті самі рядки)
ERC20 most sent token type	1	2 697
ERC20_most_rec_token_type	1	871

Найбільша група виявилася діагностично цікавою. Усі **829 акаунтів без ERC20-даних** — **шахрайські**:

Таблиця 4: Розподіл класів залежно від наявності ERC20-даних

	Клас 0	Клас 1
ERC20-дані є	85.02 %	14.98 %
ERC20-даних немає	0.00 %	100.00 %

Правило «немає ERC20-даних → шахрай» дає **precision = 1.0000** при **recall = 0.3804**: воно безпомилково виявляє понад третину всіх шахраїв.

Принципово, що «немає даних» і «нуль ERC20-транзакцій» — **різні речі**: є ще 4 399 акаунтів із явним `Total ERC20 txns = 0`, і серед них шахраїв немає зовсім. Отже, NaN тут не означає «нуль активності»; це артефакт складання датасета — для частини шахрайських акаунтів ERC20-статистику просто не зібрали. Сама **відсутність** значення виявляється сильнішим предиктором, ніж будь-яке значення.

Прийняте рішення: заповнювати пропуски медіаною всередині Pipeline і свідомо **не** додавати прапорець «значення було відсутнє». Такий прапорець підняв би метрики, але вчив би модель артефакту збору даних, а не поведінки акаунта. Наскільки результат усе одно спирається на цей артефакт — виміряно окремо в підрозділі «Внесок ERC20-ознак».

3.2.4 Кодування категоріальних ознак

Обидві рядкові колонки — назви найпопулярнішого токена ERC20. Тут два ускладнення:

1. **Висока потужність:** 304 і 466 унікальних назв. Прямий one-hot додав би 770 стовпців на 9 841 рядок, що гарантує перенавчання.
2. **Приховані плейсхолдери:** найчастіші «назви» — це '0' (4 399 разів) і ' ' — звичайний пробіл (1 191 раз). Це не токени.

Рішення: OneHotEncoder з `max_categories=11` — він лишає 10 найчастіших назв, а весь «хвіст» зводить в одну категорію `infrequent`. Важливо, що порогом керує сам енкодер **усередині Pipeline**: список популярних tokenів рахується лише на навчальній частині кожного фолда, інакше вибір категорій підглядав би в тест.

OrdinalEncoder тут не годиться: назви tokenів номінальні, у них немає природного порядку, а kNN і SVM сприймали б різницю кодів як відстань.

Після кодування маємо **60 ознак** (38 числових + one-hot від двох категоріальних).

3.2.5 Викиди

За правилом трьох сигм частка викидів невелика — максимум 1.95 % (`Avg min between received tnx`). Викиди **не видаляємо**: правило трьох сигм передбачає нормальний розподіл, а всі грошові ознаки мають важкий правий хвіст (асиметрія `total ether received` — 58.8). Це не помилки вимірювання, а саме та поведінка, яку модель має розпізнавати.

3.3 Крок 3. Візуалізація та дослідження даних (EDA)

Числових ознак 38, тому для рисунків відібрано вісім найінформативніших — за важливістю у Random Forest.



Рисунок 2: Дванадцять найінформативніших ознак

Таблиця 5: Вісім найінформативніших ознак

Ознака	Важливість
Time Diff between first and last (Mins)	0.1185
Total ERC20 txns	0.0900
ERC20 uniq rec addr	0.0776
ERC20 uniq rec contract addr	0.0674
total ether received	0.0673
ERC20 uniq rec token name	0.0637
Avg min between received txn	0.0563
max value received	0.0414

3.3.1 Кореляції

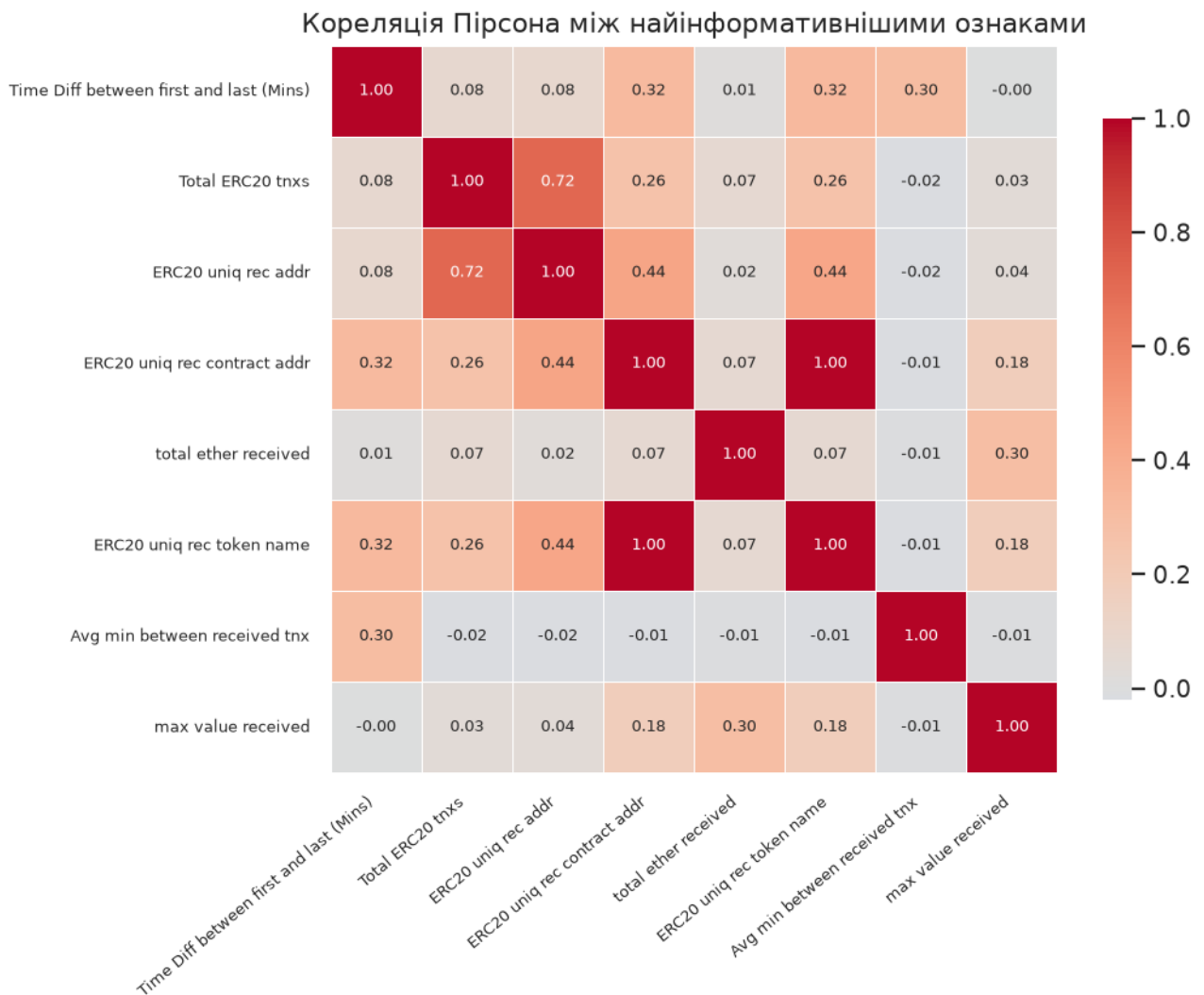


Рисунок 3: Кореляція Пірсона між найінформативнішими ознаками

Знайдено **пару з кореляцією рівно 1.000** — ERC20 uniq rec token name і ERC20 uniq rec contract addr. Це означає, що кожен отриманий токен приходив рівно з одного контракту, тож одна з колонок є точним дублікатом іншої за інформацією. Для дерев це нешкідливо, але вони «ділять» між собою важливість, через що обидві виглядають менш значущими, ніж є насправді. Наступна за силою кореляція — ERC20 uniq rec addr з Total ERC20 tnxs (0.717).

3.3.2 Гістограми

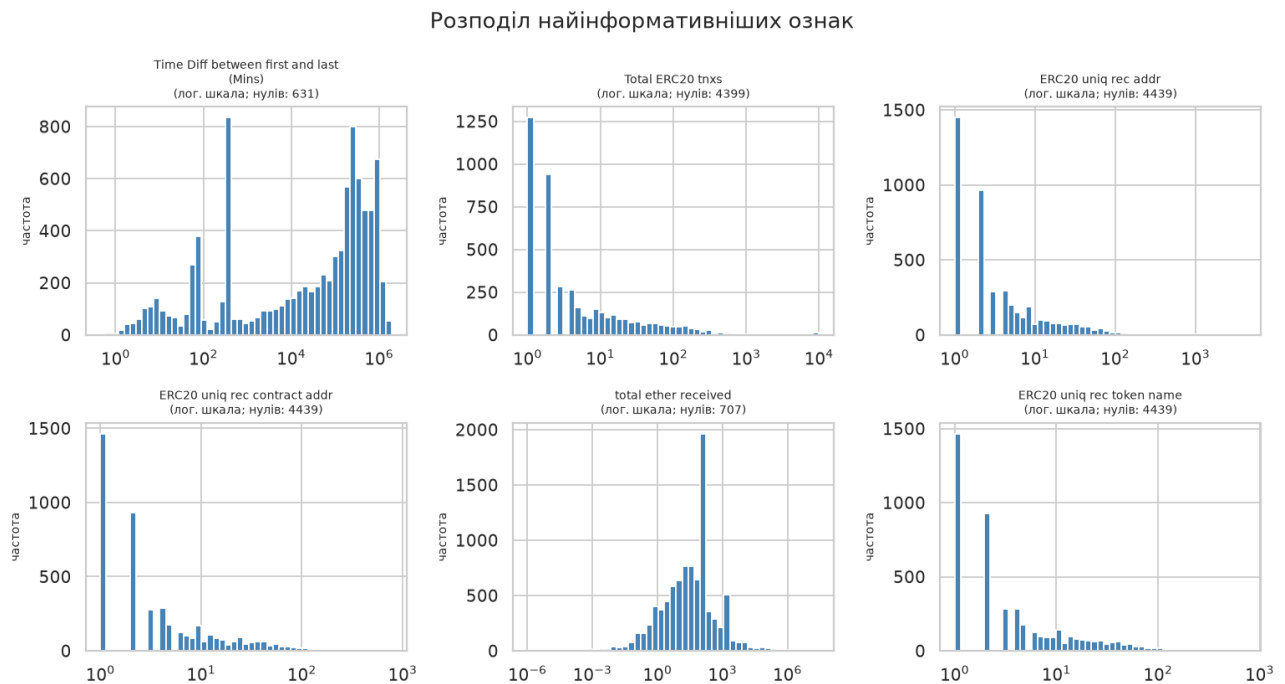


Рисунок 4: Розподіл найінформативніших ознак

Усі грошові та лічильні ознаки мають вкрай важкий правий хвіст (асиметрія від 15.5 до 58.8), тому для них застосовано логарифмічну шкалу — у лінійній «коробки» й стовпчики зливаються в одну лінію біля нуля.

На логарифмічній шкалі (з логарифмічними ж бінами) Time Diff between first and last (Mins) виявляється двомодальним: один пік — кілька сотень хвилин, другий — $10^5 \dots 10^6$ хвилин, тобто місяці й роки. Проте моди **не збігаються з класами**, як можна було б очікувати:

Таблиця 6: Частка шахраїв залежно від часу життя акаунта

Час між першою та останньою транзакцією	Акаунтів	Шахраїв
рівно 0 (усе в одному блоці)	631	92.4 %
до 10^3 хв (до ~17 год)	2 773	5.2 %
$10^3 \dots 10^5$ хв (~17 год ... ~2 міс.)	2 101	50.6 %
понад 10^5 хв	4 336	8.9 %

Короткоживучі акаунти переважно легітимні, а шахраї зосереджені у двох місцях: серед акаунтів із нульовим часом життя та в проміжному діапазоні від доби до двох місяців. Залежність від часу життя, отже, **немонотонна** — і саме таку залежність дерева рішень виражають природно, кількома порогами.

3.3.3 Розподіл за класами

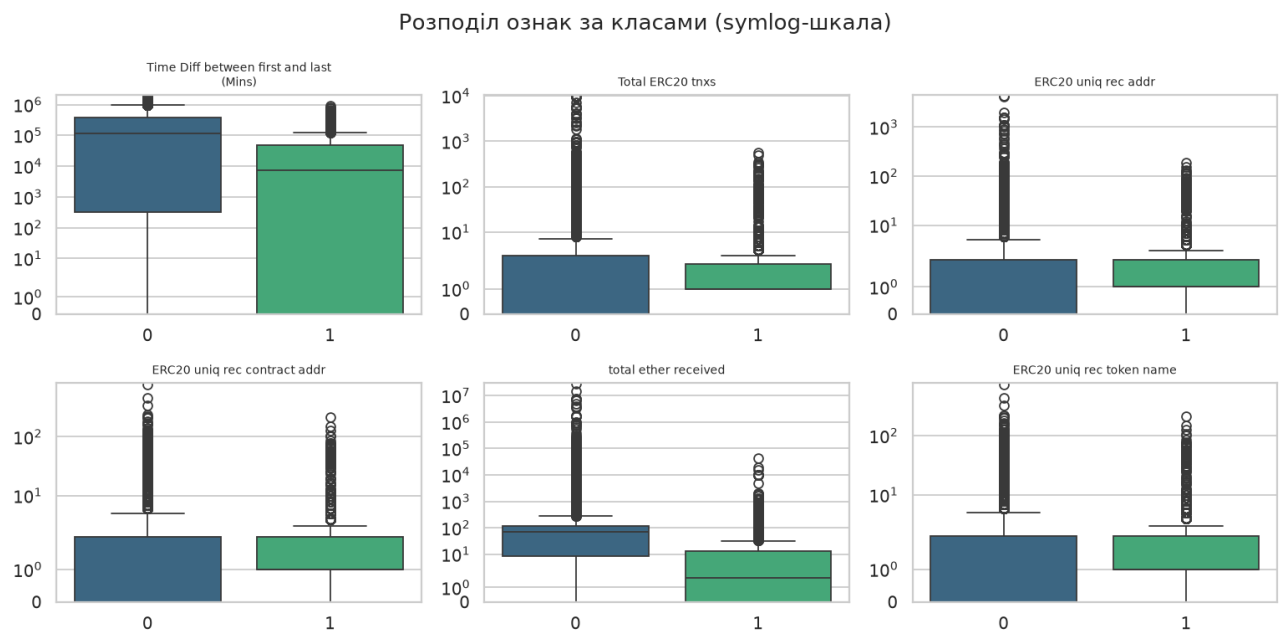


Рисунок 5: Розподіл ознак за класами

Медіани за класами дають найяскравіший результат усього EDA:

Таблиця 7: Медіани ознак за класами

Ознака	Клас 0	Клас 1
Time Diff between first and last (Mins)	116 626	7 545
total ether received	71.72	1.68
Total ERC20 txns	0	1
ERC20 uniq rec addr	0	1

Портрет шахрайського акаунта проступає прямо з цих чисел: він **живе на порядок менше** (7.5 тис. хвилин проти 117 тис., тобто приблизно 5 днів проти 81), отримує **на два порядки менше ЕТН** (1.7 проти 71.7) і при цьому має ненульову ERC20-активність там, де в медіанного легітимного акаунта її немає зовсім. Це узгоджується зі звичайним життєвим циклом скам-адреси: швидко створена, швидко використана, швидко покинута.

3.4 Крок 4. Підготовка даних до навчання

Підвибірка не потрібна: 9 841 рядок — це той розмір, на якому навіть SVM із квадратичною складністю рахується за частки секунди, тож усі моделі навчаються на повних даних.

Поділ стратифікований, у пропорції 75 / 25: **7 380** записів train і **2 461** test. Стратифікація зберігає пропорції класів із точністю до 0.0001 (0.7786 / 0.2214 проти 0.7785 / 0.2215 на тесті).

3.4.1 Масштабування

StandardScaler необхідний **kNN** і **SVM**: обидва працюють з відстанями, а ознаки тут різняться на порядки — хвилини між транзакціями (до 1.95 млн) проти сум у ЕТН. Деревам (DecisionTree, RandomForest, AdaBoost) масштабування не потрібне: вони розбивають ознаки за порогами, і монотонне перетворення нічого не змінює.

Увесь препроцесинг зібрано **всередині Pipeline**. Це принципово: і медіана для заповнення пропусків, і середнє зі стандартним відхиленням для масштабування, і список популярних токенів рахуються лише на навчальній частині кожного фолда. Інакше статистика тесту потрапила б у навчання й оцінка була б завищеною.

3.5 Крок 5. Навчання моделей

Таблиця 8: Моделі з параметрами за замовчуванням

Модель	Accuracy	Precision	Recall	F1-macro	Навчання, с
AdaBoost	0.9935	0.9959	0.9853	0.9905	3.00
Random Forest	0.9919	0.9948	0.9817	0.9881	0.80
Decision Tree	0.9866	0.9803	0.9809	0.9806	0.10
kNN	0.9854	0.9865	0.9709	0.9784	0.06
SVM (RBF)	0.9825	0.9854	0.9638	0.9741	0.53

Час навчання тут і далі — орієнтир з одного прогону; він залежить від завантаженості процесора, тому порівнювати варто порядки величин. Метрики якості, на відміну від часу, відтворюються точно.

Усі моделі істотно перевершують базову лінію 0.7786. Ансамблі (AdaBoost, Random Forest) одразу попереду, розрив між найкращою та найгіршою моделлю — лише 0.016 F1.

3.6 Крок 6. Підбір гіперпараметрів

3.6.1 kNN — n_neighbors

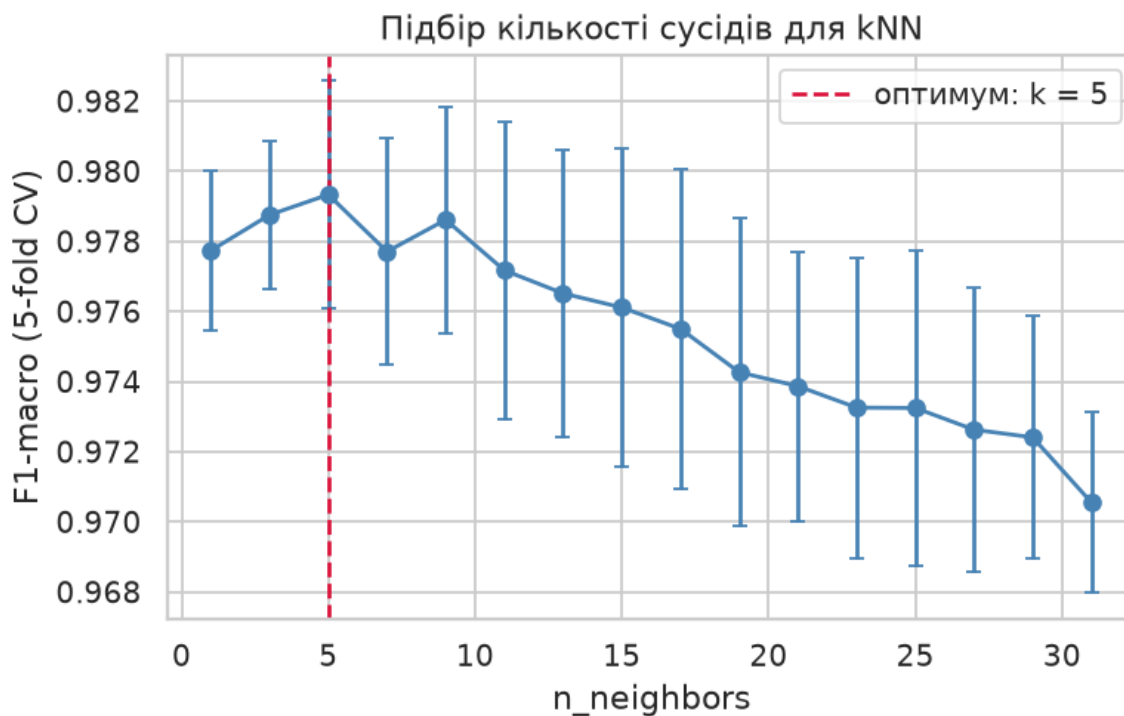


Рисунок 6: Підбір кількості сусідів для kNN

Перебрано непарні значення від 1 до 31 за 5-fold крос-валідацією. Оптимум — **n_neighbors = 5** ($F1 = 0.9793$), і цього разу він **внутрішній**: $k = 1$ дає 0.9777, $k = 3$ — 0.9788, $k = 7$ — 0.9777. Крива має виражений максимум, а не плато, тож вибір k тут осмислений, а не довільний. Далі, зі зростанням k , якість монотонно спадає до 0.9706 при $k = 31$: надто велике сусідство починає «розмивати» межу між класами.

3.6.2 SVM — GridSearchCV по C та gamma

Сітка значень: $C = \{0.1, 1, 10, 100, 1000\}$, $\gamma = \{0.001, 0.01, 0.1, 1\}$. Разом 20 комбінацій $\times$ 5 фолдів = 100 навчань, 14 с.

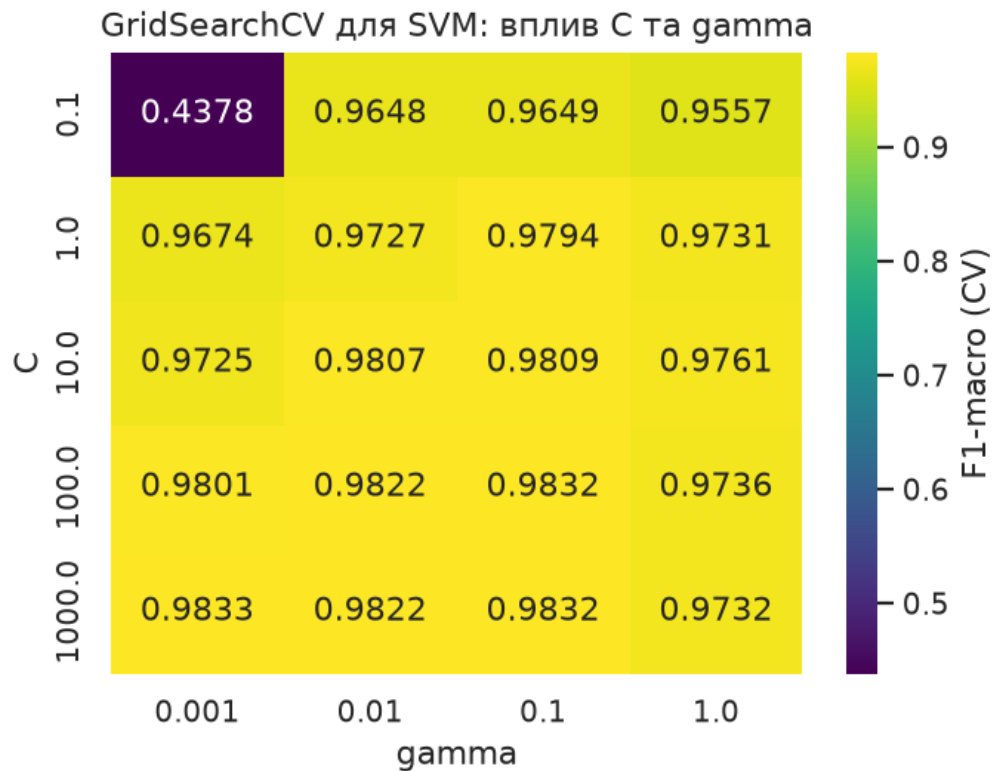


Рисунок 7: GridSearchCV для SVM

Таблиця 9: Результати GridSearchCV (F1-macro на крос-валідації)

C (рядки) / gamma (стовпці)	0.001	0.01	0.1	1.0
0.1	0.4378	0.9648	0.9649	0.9557
1	0.9674	0.9727	0.9794	0.9731
10	0.9725	0.9807	0.9809	0.9761
100	0.9801	0.9822	0.9832	0.9736
1000	0.9833	0.9822	0.9832	0.9732

Найкраща комбінація — $C = 1000$, $\text{gamma} = 0.001$ ($F1 = 0.9833$).

Ліва верхня клітинка показує: при $C = 0.1$ і $\text{gamma} = 0.001$ якість падає до **0.4378**. Це майже точно значення F1-macro для моделі, яка завжди відповідає «легітимний» — надмірна регуляризація в поєднанні з надто широким ядром повністю «здає» менший клас. Решта таблиці — широке плато 0.980...0.983, де різниці лежать у межах похибки крос-валідації.

Видно й типовий для RBF-ядра компроміс: оптимальна gamma зменшується зі зростанням C (при $C = 100$ найкраще $\text{gamma} = 0.1$, при $C = 1000$ — $\text{gamma} = 0.001$). Обидва параметри керують складністю межі й частково компенсують один одного.

3.6.3 Внесок ERC20-ознак: скільки якості дає артефакт

У підрозділі про пропуски з'ясувалося, що сама відсутність ERC20-даних безпомилково вказує на шахрая. Щоб оцінити, наскільки результат тримається на цьому артефакті, порівняно три набори ознак на однакових Random Forest (5-fold CV на навчальній вибірці).

Таблиця 10: Внесок груп ознак

Набір ознак	Числових	F1-macro (CV)
Усі ознаки	38	0.9862 ± 0.0027
Без ERC20-ознак	22	0.9121 ± 0.0073
Лише ERC20-ознаки	16	0.9818 ± 0.0024

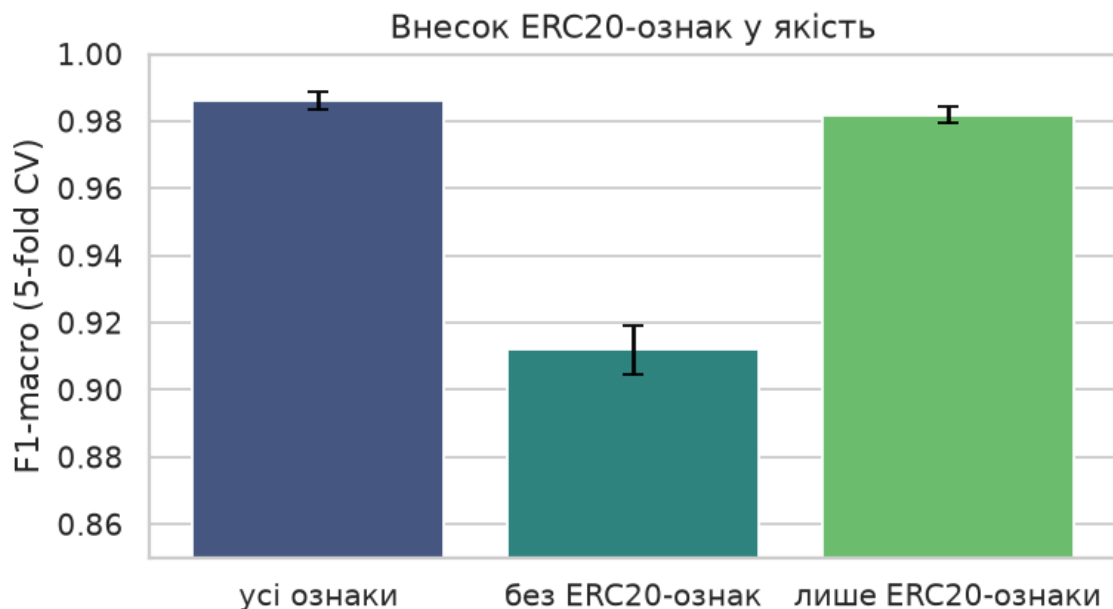


Рисунок 8: Внесок ERC20-ознак у якість

Результат відрізняється від очікуваного. Шістнадцять ERC20-ознак **майже повністю визначають якість**: самі по собі вони дають 0.9818 проти 0.9862 для всіх 38, тоді як без них якість падає до 0.9121 — тобто на 0.074. Іншими словами, 22 «поведінкові» ознаки (суми ETH, інтенсивність транзакцій, кількість контрагентів) разом слабші, ніж самі лише ERC20-ознаки.

Це прямий наслідок артефакту з пропусками: 829 шахрайських акаунтів розпізнаються за самою структурою пропусків в ERC20-колонках, і жодна поведінкова ознака з цим конкурувати не може.

Практичний висновок: оголошені ~0.99 не варто сприймати як реальну якість виявлення шахраїв. **Чеснішою оцінкою є 0.9121** — результат на ознаках, які описують саму поведінку акаунта, а не особливості складання датасета. На реальних даних, де ERC20-статистика збирається однаково для всіх акаунтів, слід очікувати саме такого порядку якості.

3.6.4 Decision Tree — max_depth

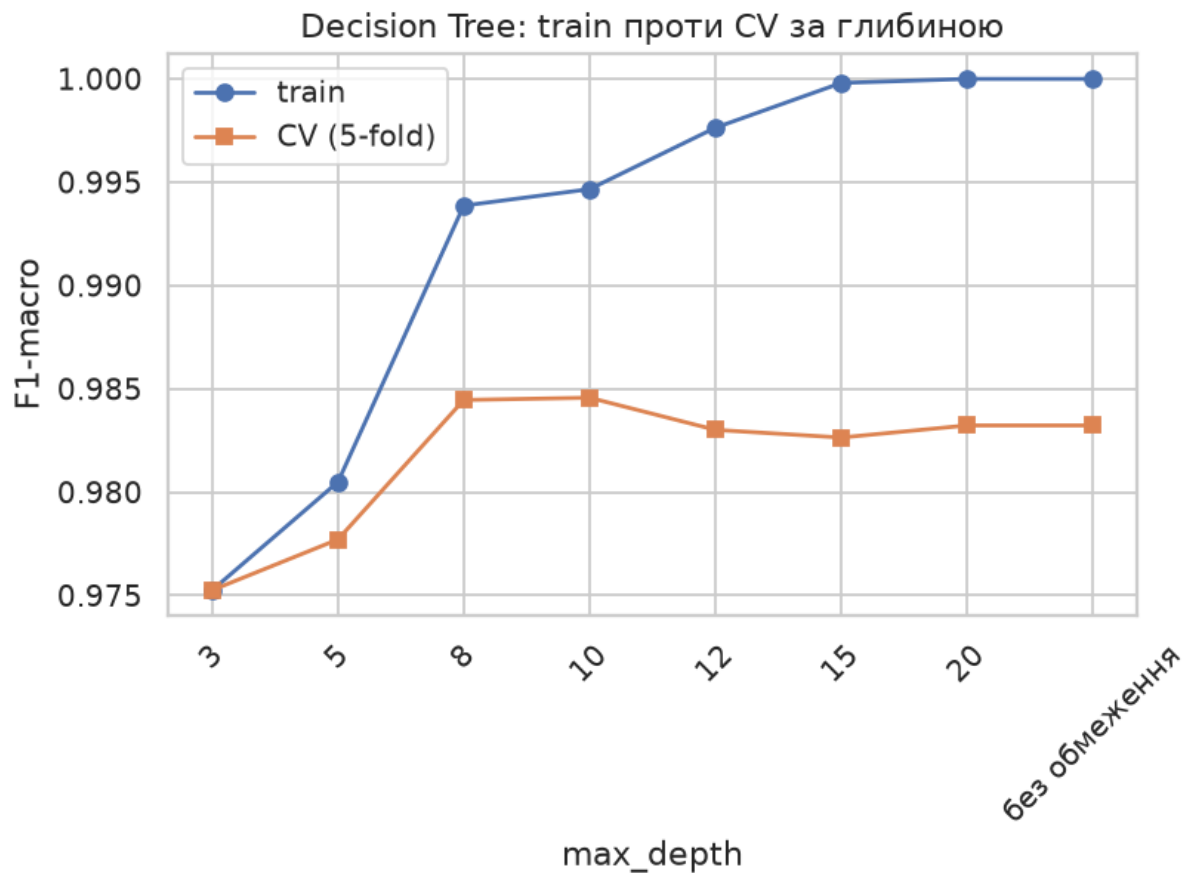


Рисунок 9: Decision Tree: train проти CV за глибиною

Максимум крос-валідації припадає на `max_depth = 10` (0.9846), після чого якість повільно спадає до 0.9832. При цьому `train`-метрика досягає рівно **1.0000** на глибині 20 і без обмеження — дерево повністю запам'ятовує навчальну вибірку. Класична картина перенавчання: розрив між `train` і `CV` зростає з 0.0000 (глибина 3) до 0.0168 (без обмеження). Обмеження глибини десятима рівнями дає і кращу якість на крос-валідації, і простішу модель.

4 Результати

Таблиця 11: Налаштовані моделі на тестовій вибірці

Модель	Параметри	Accuracy	Precision	Recall	F1-macro	Навчання, с
AdaBoost	200 оцінювачів	0.9935	0.9959	0.9853	0.9905	2.50
Random Forest	300 дерев	0.9919	0.9948	0.9817	0.9881	0.77

Модель	Параметри	Accuracy	Precision	Recall	F1-macro	Навчання, с
SVM (RBF)	C=1000, gamma=0.001	0.9874	0.9885	0.9748	0.9815	0.25
Decision Tree	типово	0.9866	0.9803	0.9809	0.9806	0.10
kNN	k = 5	0.9854	0.9865	0.9709	0.9784	0.05

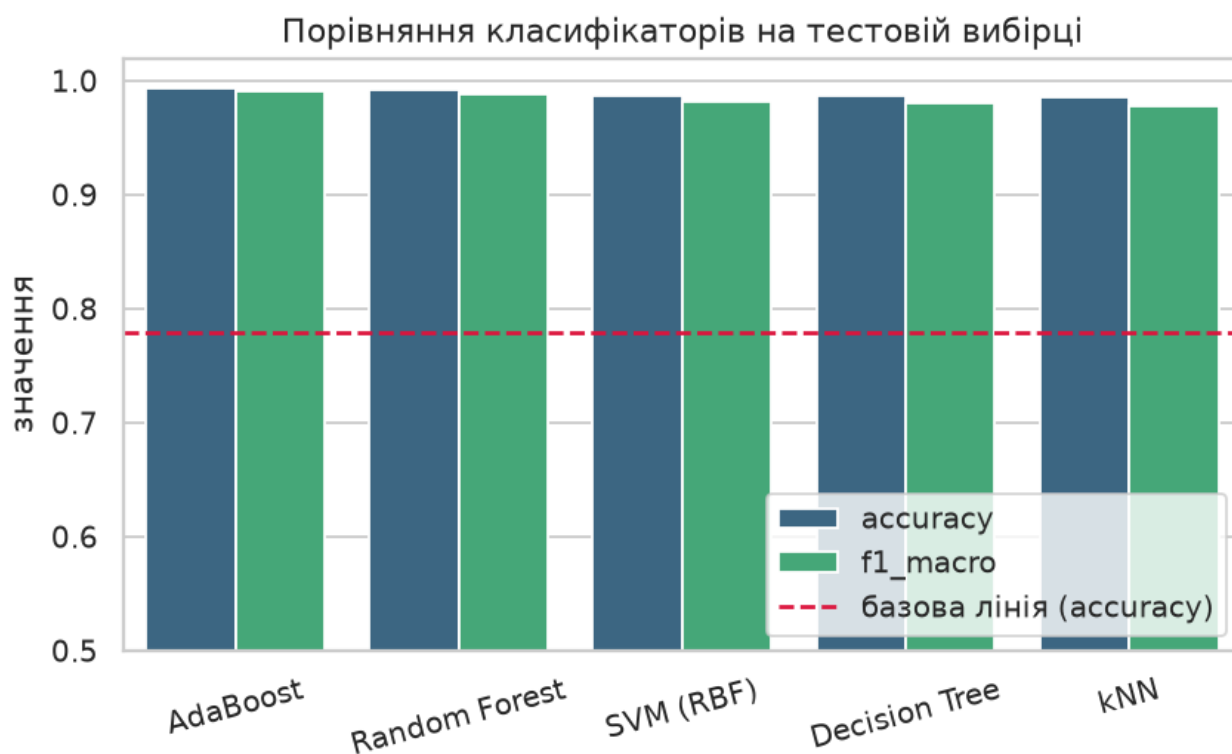


Рисунок 10: Порівняння класифікаторів

Ефект налаштування гіперпараметрів виявився скромним: SVM +0.0074 F1 (0.9741 → 0.9815), kNN 0.0000 (оптимальне k = 5 збіглося зі значенням за замовчуванням). Порядок моделей від налаштування не змінився.

Детальний звіт для найкращої моделі (AdaBoost):

	precision	recall	f1-score	support
легітимний	0.9917	1.0000	0.9958	1916
шахрайський	1.0000	0.9706	0.9851	545
accuracy			0.9935	2461
macro avg	0.9959	0.9853	0.9905	2461

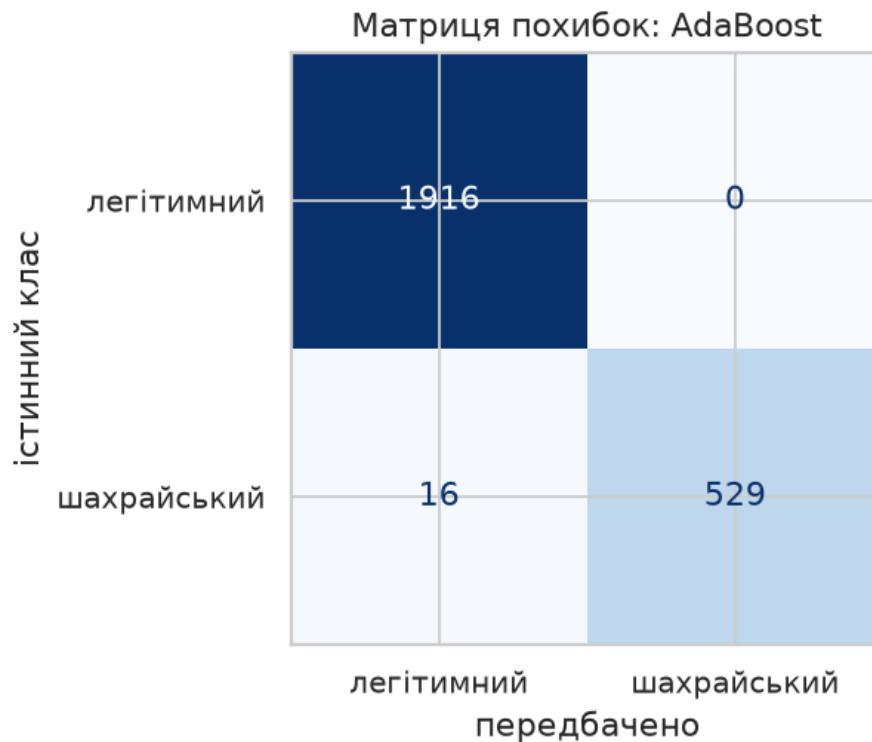


Рисунок 11: Матриця похибок для AdaBoost

Структура помилок асиметрична й сприятлива для задачі: **precision = 1.0000** на класі шахраїв означає жодного хибного звинувачення легітимного акаунта, а $recall = 0.9706$ — що пропущено 16 шахраїв із 545. Для антифроду це правильний компроміс: хибне блокування чесного користувача коштує дорожче, ніж пропуск частини порушників, яких виявлять інші механізми.

5 Висновки

За кроками роботи.

1. **Первинний аналіз.** Датасет містить 9 841 акаунт Ethereum із 50 ознаками; класи незбалансовані у співвідношенні 22.14 % / 77.86 %, базова лінія — 0.7786. Через дисбаланс основною метрикою обрано F1-macro, а не accuracy.
2. **Попередня обробка** дала головний результат усієї роботи — **виявлення витоку мітки**. Три службові колонки кодували клас безпосередньо: рядки відсортовані за класом, а Index нумерується всередині кожного класу окремо. Дерево лише на цих колонках дає $F1 = 0.9997$. Якби їх не прибрали, уся робота вимірювала б здатність моделі читати порядковий номер рядка. Додатково видалено 7 константних колонок і опрацьовано пропуски у 18 колонках.
3. **EDA** дав змістовний портрет шахрайського акаунта: він живе приблизно 5 днів проти 81 у легітимного, отримує на два порядки менше ETH і має ненульову ERC20-активність

там, де в медіанного легітимного її немає. Виявлено також пару ознак із кореляцією рівно 1.000 — точний інформаційний дублікат.

4. **Підготовка даних.** Стратифікація зберегла пропорції класів до 0.0001. Ключове технічне рішення — увесь препроцесинг (імпутація, one-hot із відсіканням рідкісних категорій, стандартизація) винесено **всередину Pipeline**, тому статистика тесту не потрапляє в навчання ні при поділі, ні при крос-валідації.
5. **Навчання моделей.** Усі п'ять класифікаторів дали F1-масро в межах 0.974...0.991 — розрив між найкращим і найгіршим лише 0.016. Попереду ансамблі: AdaBoost (0.9905) і Random Forest (0.9881).
6. **Підбір гіперпараметрів** майже нічого не змінив: SVM виграв 0.0074, а для kNN оптимальне $k = 5$ збіглося зі значенням за замовчуванням. Це саме по собі результат: коли ознаки добре розділяють класи, вибір моделі та її налаштування впливають менше, ніж якість самих ознак.

Окремі спостереження.

Головний методологічний висновок. Найцінніше, що дала робота, — не 0.9905 F1, а два знайдені дефекти даних. Витік через службові колонки зробив би роботу беззмисловою непомітно для метрик: усі показники були б **вищими**, а не нижчими. Це ілюструє загальне правило — аномально високу якість треба сприймати як привід для перевірки даних, а не як успіх.

Про справжню якість. Абляція показала, що 16 ERC20-ознак майже повністю визначають результат (0.9818 самі по собі проти 0.9862 для всіх 38), а без них якість падає до 0.9121. Оскільки частина цього внеску — артефакт складання датасета (829 шахрайських акаунтів упізнаються за самою структурою пропусків), чеснішою оцінкою реальної здатності виявляти шахраїв є **0.9121**, а не 0.99.

Чому ансамблі виграли. AdaBoost і Random Forest випередили одиночне дерево (0.9905 і 0.9881 проти 0.9806). Задача має порогову, осьову структуру («якщо акаунт живе менше N хвилин і має ERC20-активність...»), яку дерева виражають природно, а усереднення багатьох дерев прибирає дисперсію одиночного. kNN і SVM, які працюють з відстанями, опинилися позаду: у 60-вимірному просторі з переважно розрідженими one-hot стовпцями відстань гірше відображає схожість акаунтів.

Обмеження роботи. Оцінки отримано на одному розбитті 75/25 з фіксованим `random_state`; підбір гіперпараметрів виконано за 5-fold крос-валідацією на навчальній частині, тож тестова вибірка не брала участі в налаштуванні. Датасет охоплює обмежений період і конкретний набір шахрайських схем, тому перенесення моделі на свіжі дані потребує повторної перевірки.

6 Джерела

1. Ethereum Fraud Detection Dataset. URL: <https://www.kaggle.com/datasets/vagifa/ethereum-frauddetection-dataset>
2. Scikit-learn User Guide. URL: https://scikit-learn.org/stable/user_guide.html
3. Pedregosa F. et al. Scikit-learn: Machine Learning in Python // Journal of Machine Learning Research. — 2011. — Vol. 12. — P. 2825–2830.
4. Kaufman S., Rosset S., Perlich C. Leakage in Data Mining: Formulation, Detection, and Avoidance // ACM Transactions on Knowledge Discovery from Data. — 2012. — Vol. 6, No. 4.
5. Матеріали курсу «Інтелектуальний аналіз даних». URL: <https://github.com/natsakh/IAD>